Manual Técnico

St4ck Over-Pig - Compilador a RISC-V

Compiladores 2 - USAC CUNOC

Brandon Gustavo Güinac Román - 201931217

Octubre 2026

1. Introducción

St4ck Over-Pig es la segunda fase del compilador desarrollado en el Proyecto 1 (Contacto 3xtrat3rr3str3D). Traduce programas escritos en tres lenguajes (Y?, Zetariano y Pig Latin) a un código intermedio de tres direcciones (cuartetas) y de ahí a dos destinos: ensamblador RISC-V (ejecutable en RARS) y código C que usa únicamente etiquetas y goto, sin funciones definidas por el usuario. El propio programa también ejecuta el ensamblador generado con un intérprete RISC-V embebido, sin depender de ninguna herramienta externa.

Novedades de esta fase:

- Zetariano: encapsulamiento (public, private, protected y default), herencia (extends) y polimorfismo con @Override obligatorio.
- Pig Latin: tipo explícito en la declaración de objetos y objetos nulos (null).
- Backend RISC-V completo (RV32IMFD) y backend C basado en etiquetas.
- Intérprete RISC-V embebido y panel de ejecución con entrada interactiva.
- Exportación del código de tres direcciones (.c3d), del C y del ensamblador.

2. Arquitectura

El compilador es una aplicación Java 17 con Maven y ANTLR 4.13.1. El flujo es:

```
fuente (.y / .z / .pig)
   -> Lexer + Parser (ANTLR4)
   -> Pasada 1: registro de firmas (clases, estructuras, métodos)
   -> Pasada 2: análisis semántico + generación de cuartetas
   -> ModeloPrograma (bajada a un modelo de bajo nivel)
   -> GeneradorCEtiquetas  -> .c  (solo etiquetas y goto)
   -> GeneradorRiscV       -> .asm (RISC-V)
   -> InterpreteRiscV      -> ejecución interna
```

Paquete	Responsabilidad
y, zetariano, pig	Gramáticas ANTLR, registradores de firmas, analizadores/visitors y resolutores de acceso y tipo de cada lenguaje.
semantico	Tipo, símbolos, tabla de símbolos global, visibilidad, gestor de errores, Etiquetas (firmas y nombres de etiquetas).
ir	Cuarteta, TablaCuartetas y FormateadorC3D (texto del código de tres direcciones).
backend	ModeloPrograma (bajada), GeneradorCEtiquetas, GeneradorRiscV y EmisorInstruccionesRiscV.
riscv	InterpreteRiscV: ensamblador y simulador de RV32IMFD.
gui	VentanaPrincipal, EjecutorPipeline, PanelEjecucion, Resaltador, TablaSimbolosFormateador.

3. Código intermedio (cuartetas)

Cada instrucción es una cuarteta (operador, arg1, arg2, resultado). Los tres lenguajes producen el mismo repertorio de operaciones, por lo que los backends no dependen del lenguaje de origen.

Operación	Significado
+ - * / % < > <= >= == !=	Aritmética y comparación sobre temporales.
= , -u , !	Asignación, negación aritmética y lógica.
label, goto, if_true, if_false	Control de flujo. && y || se cortocircuitan con estas instrucciones; el ternario evalúa solo la rama tomada.
param, call, return	Llamadas. Los parámetros se pasan en orden y, en métodos y constructores, this va al final.
new_array, new	Creación de arreglos y de objetos (new lleva Clase#firma).
new_obj, getf, setf, classid	Bajada de objetos: reserva, lectura/escritura de campo (Clase.campo) y consulta del id de clase para el despacho virtual.
concat	Concatenación de cadenas (cualquier operando se convierte a cadena).
print, read	Salida (arg2/res "nl" indica salto de línea) y lectura.
decl	Declara el tipo de una variable local (necesario para objetos y estructuras).

4. Modelo orientado a objetos

4.1 Disposición en memoria

Un objeto es un bloque de ranuras de 8 bytes. La ranura 0 guarda el identificador de clase (0 en las estructuras de Y?/Pig Latin). Después van los campos, con los heredados primero, de modo que un campo tiene el mismo desplazamiento en la superclase y en todas sus subclases. Un campo arreglo ocupa 1 + dimensiones ranuras (puntero y tamaños por dimensión), lo que permite arreglos multidimensionales correctos.

4.2 Herencia, constructores y super

El constructor de una subclase ejecuta, en orden: el constructor de la superclase (super(...) explícito o el implícito), los inicializadores de campos, y el cuerpo. Las llamadas super.metodo() y super(...) son no virtuales (enlace estático).

4.3 Polimorfismo y despacho virtual

La clase estática del receptor determina el método candidato. Si hay más de una implementación posible entre sus subclases, se emite una cadena de comparaciones sobre classid con etiquetas Ldv<N> que salta a la implementación concreta; si solo existe una, la llamada es directa. @Override es obligatorio: marcarlo sin que exista un método heredado con la misma firma es un error, y sobreescribir un método heredado (no private) sin marcarlo también.

4.4 Sobrecarga

Etiquetas.firma() une los nombres de tipo de los parámetros con "__". Las etiquetas son metodo_Clase_nombre_n y ctor_Clase_n; solo si existe otra sobrecarga con el mismo nombre y aridad se agrega __firma. Los destinos de call llevan recv.nombre#firma.

4.5 Encapsulamiento

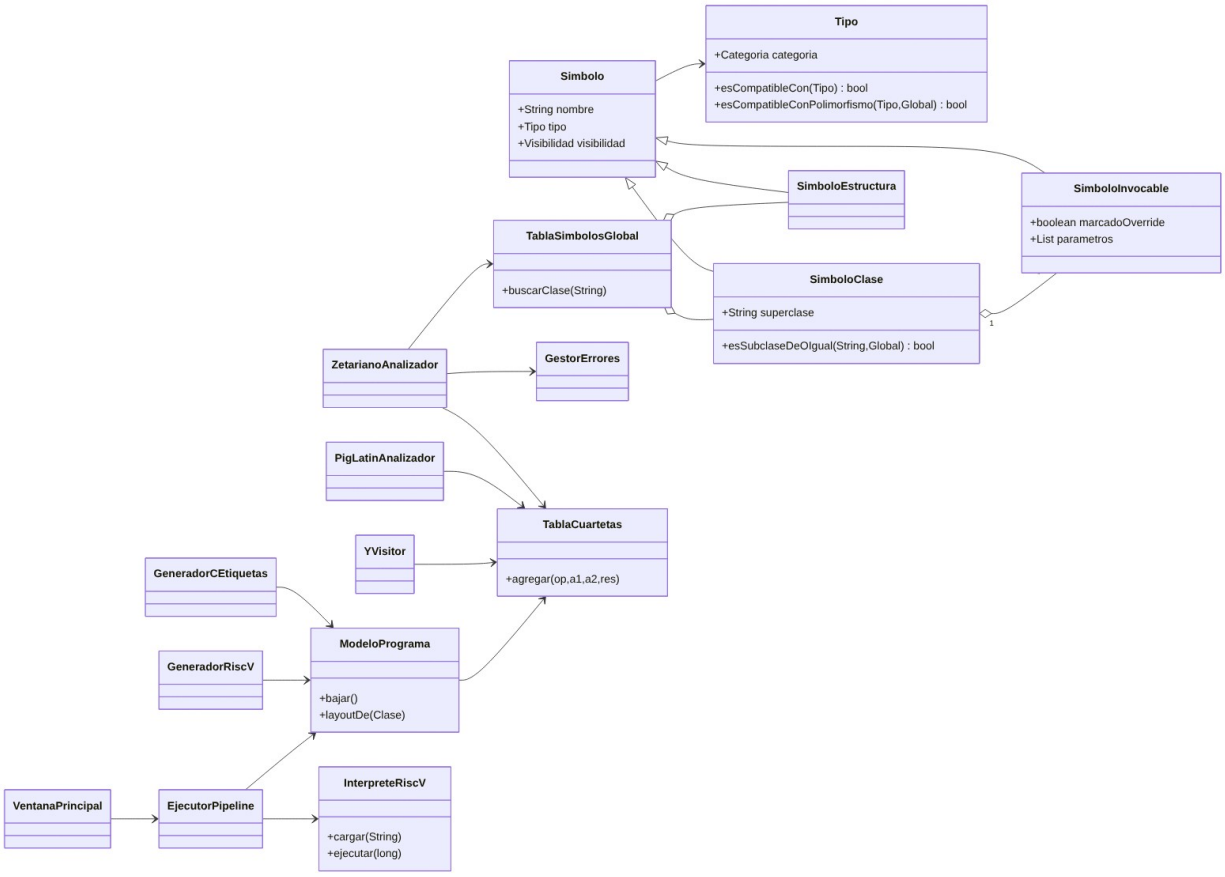
Modificador	Misma clase	Mismo paquete	Subclase (otro	Otra clase (otro

			paquete)	paquete)
public	Sí	Sí	Sí	Sí
protected	Sí	Sí	Sí	No
default (ninguno)	Sí	Sí	No	No
private	Sí	No	No	No

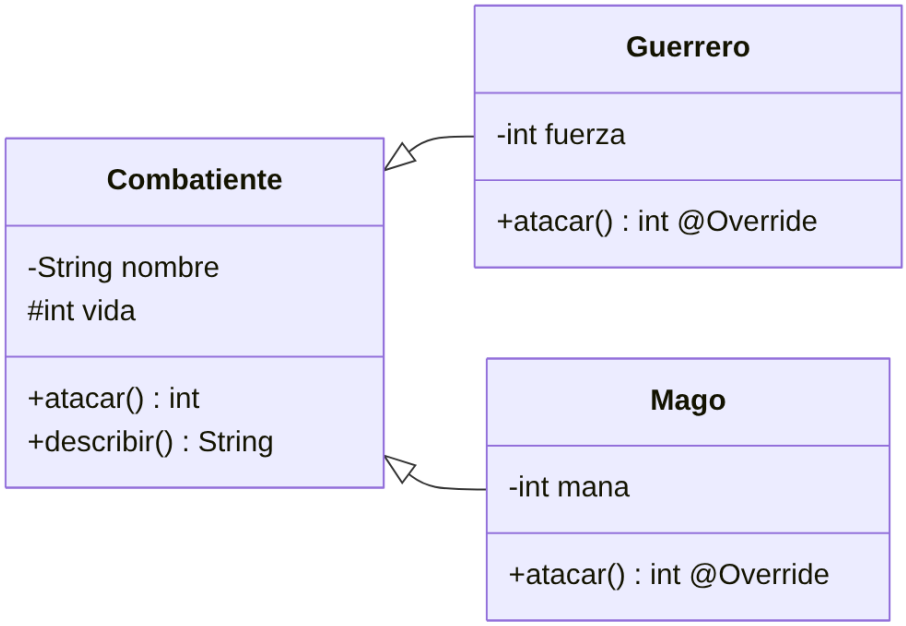
El paquete de una clase es la carpeta desde la que se importa (por ejemplo, import paquete_a.UnidadBase.z hace que UnidadBase pertenezca al paquete paquete_a; los archivos junto al .pig pertenecen al paquete raíz). Las verificaciones de visibilidad entre clases Zetariano se hacen en el análisis semántico y reportan un error con línea y columna. Desde Pig Latin, los miembros private y protected no son accesibles.

Constructor delegado: un constructor puede comenzar con this(...) para invocar a otro constructor de la misma clase; en ese caso no se repiten super() ni los inicializadores de campos.

4.6 Diagrama de clases del compilador



Ejemplo de jerarquía de un programa Zetariano (Combatiente, Guerrero, Mago), donde atacar() se sobreescribe y se resuelve en ejecución según el objeto real:



5. Tabla de compatibilidad de tipos

Las reglas están implementadas en Tipo.esCompatibleCon() y Tipo.esCompatibleConPolimorfismo(). Filas: tipo destino; columnas: tipo del valor asignado. "Sí" = asignación permitida; "Prom." = promoción implícita; "No" = error semántico.

Destino \ Valor	int	double	char	boolean	String	null	Subclase
int	Sí	No	No	No	No	No	-
double	Prom.	Sí	No	No	No	No	-
char	No	No	Sí	No	No	No	-
boolean	No	No	No	Sí	No	No	-
String	No	No	No	No	Sí	No	-
Clase / struct	No	No	No	No	No	Sí	Sí (solo clases)
Arreglo	No	No	No	No	No	Sí	Sí si mismas dimensiones

5.1 Operadores

Operador	Operandos válidos	Resultado
+ - * / %	int/double (ambos numéricos)	double si alguno es double; si no, int
+ con String	Si alguno de los lados es String	String (el otro se convierte)
+= con String	Destino String	String (concatenación)
< > <= >=	int/double	boolean
== !=	Tipos compatibles entre sí (incluye null)	boolean

Operador	Operandos válidos	Resultado
&& || !	boolean	boolean (cortocircuito)
? :	Condición boolean; ramas de tipo compatible	Tipo de las ramas

5.2 Equivalencia de tipos entre los tres lenguajes

Los tres lenguajes comparten el mismo sistema de tipos interno; solo cambia la palabra con que se escribe cada tipo. Las reglas de compatibilidad de la tabla 5 aplican igual a los tres.

Tipo interno	Y?	Zetariano	Pig Latin	Representación
Entero	entero	int	numerus	32 bits con signo
Decimal	flotante	double	decimalis	doble precisión
Carácter	caracter	char	littera	1 byte
Booleano	bool (verdadero / falso)	boolean (true / false)	bool (verum / falsus)	0 / 1
Cadena	cadena	String	textum	puntero a texto terminado en 0
Estructura	estructura (por referencia)	-	IDENTIFICADOR { ... }	bloque en heap
Objeto	-	IDENTIFICADOR (clase)	novus Clase(...) : Clase	bloque en heap, slot 0 = id de clase
Arreglo	tipo nombre[n]	tipo[] / new tipo[n]	series nombre[n] : tipo	longitud + elementos
Nulo	-	null	null	0

5.3 Reglas de compatibilidad por lenguaje

- Y?: entero a flotante se promueve; cualquier otra mezcla (cadena con número, bool con número) es error semántico. No hay objetos ni null.
- Zetariano: además de las reglas de la tabla, un objeto de una subclase se puede asignar a una variable de su clase base (polimorfismo) y null se acepta en clases, arreglos y String. Los métodos sobreescritos requieren @Override.
- Pig Latin: la declaración con tipo explícito (esto x : novus Profesor(...) : Persona ;) exige que Profesor sea la misma clase o una subclase de Persona; esto x : null : Persona ; deja el objeto nulo. Las operaciones aritméticas siguen la tabla de operadores.

6. Backend RISC-V

- Todas las variables y temporales viven en el marco de pila; t0-t2 son registros de trabajo.
- Los objetos son punteros de 32 bits; objetos, arreglos y cadenas se reservan con sbrk (ecall 9).
- Rutinas de apoyo que se añaden solo si se usan: __itoa, __ctoa, __dtoa (6 decimales, igual que %f), __concat, __strcmp y __leercad.
- La lectura de cadenas usa la llamada 8 (read string) y toma la primera palabra de la línea, igual que scanf("%255s") en el backend C.

- Los booleanos se imprimen como 1/0.

7. Backend C con etiquetas

Se genera una única función main. Las llamadas se simulan con pilas globales (pila_i, pila_d, pila_p, pila_ret) y goto a etiquetas; los parámetros pasan por registros de intercambio (sa_*) y los retornos por ret_*. Las variables se nombran <funcion>__<variable>. Los objetos son bloques char* accedidos como *(T*)(obj + 8*ranura). No hay funciones definidas por el usuario.

8. Intérprete RISC-V embebido

La clase com.company.riscv.InterpreteRiscV ensambla y simula RV32IMFD: expande pseudoinstrucciones (li, la, mv, j, call, etc.), resuelve etiquetas y emula memoria paginada (datos en 0x10010000, montículo en 0x10040000, pila en 0x7fffeffc). Syscalls soportadas: 1, 2, 3, 4, 5, 7, 8, 9, 10, 11, 12, 17, 34 y 36. Detecta accesos a la región nula, aplica un límite de instrucciones (2 000 millones) y se puede cancelar desde la GUI. Su salida fue contrastada con RARS en todos los programas de prueba.

9. Pruebas realizadas

Cada programa se compiló a C (gcc) y a RISC-V, y se ejecutó en gcc, en el intérprete embebido y en RARS, con la misma entrada, comparando la salida carácter a carácter.

Programa	Qué cubre
Estructuras	Estructuras y arreglos (Y?/Pig Latin).
Control	if, switch, for, while, do-while, break, continue.
Vida	Arreglos bidimensionales.
Arena	Clases, herencia, polimorfismo, entrada de usuario.
Laberinto	Objetos nulos, cortocircuito, arreglos de objetos, entrada de usuario.
TicTacToe	Matrices y lógica de juego.
OOP propio	double, arreglos de objetos, cadenas de super, sobrecarga.
05_errores	Errores semánticos esperados (visibilidad, @Override, tipos).

10. Limitaciones conocidas

- Los valores de estructura se pasan por referencia (no se copian).
- Los booleanos se imprimen como 1 y 0.
- Una cadena nula (por ejemplo un atributo String sin inicializar) se imprime y se concatena como "null", igual que en Java.
- El C generado puede producir advertencias de etiquetas o variables sin uso con -Wall; no afectan la ejecución.
- La profundidad máxima de recursión en C depende del tamaño de las pilas globales (4 194 304 entradas).

11. Tecnologías utilizadas

Tecnología	Uso
Java 17	Lenguaje de implementación del compilador y de la interfaz.
ANTLR 4.13.1	Generación de los analizadores léxico y sintáctico de Y?, Zetariano y Pig Latin (patrón Visitor).
Maven	Construcción del proyecto y descarga de dependencias.
Swing (Java)	Interfaz gráfica: editor, árbol de archivos, pestañas y consola de ejecución.
Resaltador propio (Resaltador.java)	Resaltado de sintaxis en tiempo real de los tres lenguajes, sin librerías externas.
RISC-V RV32IMFD	Arquitectura de destino; el ensamblador generado es compatible con RARS.
Intérprete RISC-V propio	Ejecuta el ensamblador generado dentro de la aplicación.
C (solo etiquetas y goto)	Segundo destino de la traducción, para comparar contra el ensamblador.
RARS	Simulador externo usado para validar el ensamblador generado.
Git / GitHub	Control de versiones y entrega del código y de los manuales.

12. Especificación léxica y gramatical

12.1 Palabras reservadas

Lenguaje	Palabras reservadas
Y?	estructura, definir, entero, cadena, flotante, caracter, bool, verdadero, falso, si, entonces, sino, contrario, elegir, caso, siempre, para, mientras, hacer, romper, continuar, retornar, %estructuras, %funciones
Zetariano	public, private, protected, class, extends, void, int, double, char, boolean, String, new, this, super, null, true, false, if, else, switch, case, default, for, while, do, break, continue, return, @Override
Pig Latin	import, VARIABILES>, MAIOR>, FINIS, esto, series, numerus, textum, decimalis, littera, bool, verum, falsus, null, novus, si, aliter, finis, dum, facere, per, perge, interrumpe

12.2 Símbolos y operadores

Categoría	Símbolos	Lenguajes
Aritméticos	+ - * / %	Los tres (Y? sin %)
Relacionales	< > <= >= == !=	Los tres
Lógicos	&& || !	Los tres
Asignación	=	Los tres
Asignación compuesta	+= -= *= /= %=	Zetariano
Incremento / decremento	++ --	Los tres
Ternario	? :	Zetariano
Acceso	. [ ] ( )	Los tres
Delimitadores	{ } ; , :	Los tres
Flecha de retorno	->	Y?

Categoría	Símbolos	Lenguajes
Entrada / salida	>> <<	Pig Latin
Anotación	@	Zetariano
Comentarios	// línea /* bloque */ ## … ##	// y /* */ en los tres; ## solo Pig Latin

12.3 Gramática de Zetariano (resumen)

```
programa     : 'public'? 'class' ID ('extends' ID)? '{' miembro* '}'
miembro      : campo | constructor | metodo
campo        : visibilidad? tipo ID ('=' inicializador)? ';'
visibilidad  : 'public' | 'private' | 'protected'   // sin modificador = default (paquete)
constructor  : visibilidad? ID '(' params? ')' bloque
metodo       : ('@' ID)? visibilidad? (tipo | 'void') ID '(' params? ')' bloque
tipo         : ('int'|'double'|'char'|'boolean'|'String'|ID) ('[' ']')*
sentencia    : declaracion ';' | asignacion ';' | incDec ';' | condicional
             | switch | for | while | do-while | 'break' ';' | 'continue' ';'
             | 'return' expr? ';' | llamada ';' | bloque
asignacion   : acceso ('='|'+='|'-='|'*='|'/='|'%=') expr
acceso       : (ID | 'this' | 'super') ( '.' ID | '[' expr ']' | '(' args? ')' )*
expr         : expr op expr | ('!'|'-') expr | expr '?' expr ':' expr
             | 'new' ID '(' args? ')' | 'new' tipo ('[' expr ']')+
             | '(' expr ')' | acceso | literal
```

12.4 Gramática de Pig Latin (resumen)

```
programa      : importacion* ('VARIABILES>' declaracion*)? 'MAIOR>' sentencia* 'FINIS' ';'
importacion   : 'import' ID ('.' ID)*
declaracion   : 'esto' ID ':' inicializador (':' ID)? ';'
              | 'esto' ID ':' ID ';'
              | 'series' ID ('[' expr ']')+ ':' tipoDato valores? ';'
inicializador : tipoPrimitivo expr | 'bool'? ('verum'|'falsus')
              | ID '{' valores '}' | 'novus' ID '(' args? ')' | 'null'
sentencia     : declaracion | asignacion | condicional | dum | facere | per
              | 'interrumpe' ';' | 'perge' ';' | '>>' expr ('>>' expr)* ';'
              | acceso? '<<' ';' | incDec ';' | llamada ';'
condicional   : 'si' '(' expr ')' cuerpo ('aliter' '(' expr ')' cuerpo)* ('aliter' cuerpo)? 'finis' ';'
asignacion    : acceso '=' expr (':' ID)? ';'
```

12.5 Gramática de Y? (resumen)

```
programa      : ('%estructuras' estructura+)? '%funciones' funcion+
estructura    : 'estructura' ID ':' INDENT (tipo ID ('[' expr ']')?)+ DEDENT
funcion       : 'definir' ID '(' params? ')' ('->' tipo)? ':' INDENT sentencia+ DEDENT
param         : '[' ']' tipo ID | '{' '}' tipo ID | tipo ID
sentencia     : declaracion | asignacion | incDec | 'romper' | 'continuar'
              | 'retornar' expr? | llamada | condicional | elegir | para | mientras | hacer
condicional   : 'si' '(' expr ')' 'entonces' bloque ('sino' '(' expr ')' 'entonces' bloque)*
                ('contrario' bloque)?
elegir        : 'elegir' '(' expr ')' ':' ('caso' literal ':' bloque | 'siempre' ':' bloque)+
para          : 'para' '(' init? ';' expr? ';' act? ')' ':' bloque
mientras      : 'mientras' '(' expr ')' 'hacer' bloque
hacer         : 'hacer' ':' bloque 'mientras' '(' expr ')'
```

La gramática completa está en los archivos YLexer.g4, YParser.g4, Zetariano.g4 y PigLatin.g4 del repositorio. Los bloques de Y? se delimitan por indentación (tokens INDENT/DEDENT generados por el analizador léxico).